VRhook: A Data Collection Tool for VR Motion Sickness Research

Elliott Wen*, Tharindu Kaluarachchi*, Shamane Siriwardhana*, Vanessa Tang*, Mark Billinghurst*, Robert W. Lindeman§, Richard Yao‡, James Lin‡, Suranga Nanayakkara†*,

AH Lab, The University of Auckland, New Zealand* HIT Lab NZ, University of Canterbury, New Zealand§
Facebook, San Francisco, California, United States‡ Dept of Information Systems Analytics, NUS, Singapore†
{elliott,tharindu,shamane,vanessa,suranga}@ahlab.org,mark.billinghurst@auckland.ac.nz
gogo@hitlabnz.org,{richard.yao,james.lin}@fb.com

ABSTRACT

Despite the increasing popularity of VR games, one factor hindering the industry's rapid growth is motion sickness experienced by the users. Symptoms such as fatigue and nausea severely hamper the user experience. Machine Learning methods could be used to automatically detect motion sickness in VR experiences, but generating the extensive labeled dataset needed is a challenging task. It needs either very time consuming manual labeling by human experts or modification of proprietary VR application source codes for label capturing. To overcome these challenges, we developed a novel data collection tool, VRhook, which can collect data from any VR game without needing access to its source code. This is achieved by dynamic hooking, where we can inject custom code into a game's run-time memory to record each video frame and its associated transformation matrices. Using this, we can automatically extract various useful labels such as rotation, speed, and acceleration. In addition, VRhook can blend a customized screen overlay on top of game contents to collect self-reported comfort scores. In this paper, we describe the technical development of VRhook, demonstrate its utility with an example, and describe directions for future research.

CCS CONCEPTS

• **Information systems** → *Computing platforms*; • **Computing methodologies** → **Machine learning**; • **Software and its engineering** → *Software creation and management*.

KEYWORDS

Automatic Data Collection, Motion Sickness, VR, Machine Learning

ACM Reference Format:
Elliott Wen[1], Tharindu Kaluarachchi[1], Shamane Siriwardhana[1], Vanessa Tang[1], Mark Billinghurst[1], Robert W. Lindeman[4], Richard Yao[3], James Lin[3], Suranga Nanayakkara[2][1],. 2022. VRhook: A Data Collection Tool for VR Motion Sickness Research. In *The 35th Annual ACM Symposium on User Interface Software and Technology (UIST '22), October 29-November 2, 2022, Bend, OR, USA.* ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/3526113.3545656

UIST '22, October 29-November 2, 2022, Bend, OR, USA

ACM ISBN 978-1-4503-9320-1/22/10…$15.00
https://doi.org/10.1145/3526113.3545656

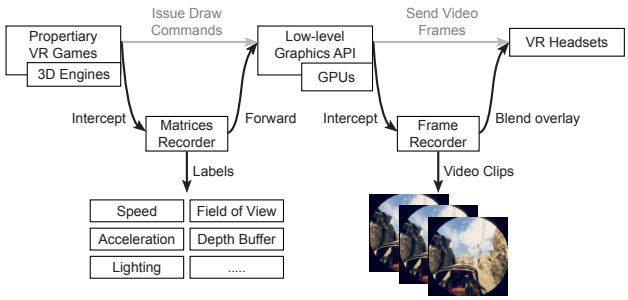

Figure 1: Backbone techniques of our data collection tool.

1 INTRODUCTION

In this paper we report on a new tool that can help with identifying Virtual Reality (VR) gaming experiences that are likely to cause motion sickness. VR gaming has been gaining widespread popularity in recent years, with the annual market revenue projected to reach $84 billion by 2028 [1]. However, up to 40% of users suffer from VR motion sickness with symptoms like fatigue, disorientation, and nausea [2, 3]. These adverse effects can severely undermine the user experience. To inform users about potential motion sickness, VR game stores like Oculus and Steam display a comfort rating for each game. These comfort ratings are determined by human experts who can identify common risk factors inside the application, such as frequent multi-axis rotation, excessive movement speed, and use of wide field of view [4]. However, a significant drawback of this approach is the highly labor-intensive rating process, which does not scale with the growing game industry.

Recently, researchers have proposed the use of machine learning approaches to identify the presence of motion sickness [5, 6]. Despite the progress made, many of these studies pointed out that their models are constrained by the size of training datasets. To improve the model generalization, they need to acquire larger datasets, containing many thousands of video clips of VR experiences and the corresponding risk factors [7]. Unfortunately, conventional data acquisition strategies cannot meet this requirement. Manual annotation by human experts is expensive and does not scale. An alternative is to use custom-made VR games for data collection, however this could create generalizability issues. Finally, modifying existing VR games to extract the data would require source code access, which is practically impossible due to the proprietary nature of the games.

To overcome these challenges, we present a novel data collection tool named VRhook, which can automatically extract labeled data from a wide range of real-world VR games without accessing

their source codes (see Figure 1). This is achieved via *Dynamic Hooking* [8], where we inject custom code into run-time memory to intercept low-level graphics pipeline data, particularly video frames and their associated transformation matrices. In computer graphics, transformation matrices apply motion effects to 3D models and project them into a two-dimensional video frame for presentation. The matrices thus can be used to extract many useful labels such as rotation, speed, and acceleration, which have been linked to motion sickness [9, 10]. In addition to data capturing, our tool can inject additional rendering commands to display in-game overlay information. This allows us to incorporate a previously validated dial mechanism [11] to collect self-reported comfort scores from users. By combining the extracted labels, the self-reported comfort scores, and the video frames, we can construct many motion sickness detection models from prior work. In this way, our tool could stimulate new machine learning based research on VR sickness.

In this paper, we present the implementation details of VRhook. We also demonstrate the utility of the tool by developing a machine learning-based pipeline to detect the presence of factors that contribute to motion sickness and predict the comfort score for a given gameplay video. The main novelty of this work is to provide a scalable method of gathering labeled data from real-world VR games without access their source code. We envision our tool can stimulate new machine learning based research on VR sickness.

2 RELATED WORK

2.1 VR Sickness and Risk Factors

Despite the advances in VR technology, users still frequently experience motion sickness from its use. Eye fatigue, nausea, and disorientation are among the most common symptoms, which can negatively impact the user experience [12]. To date, a considerable amount of medical research has attempted to uncover the causes [4, 13]. Their results unanimously suggest that VR content is the most influential factor for VR sickness. Here, we provide an overview of well known motion sickness risk factors in VR content.

Camera Speed: Humans are more likely to experience nausea when they see moving rather than static visual content [10, 14, 15].

Lo et al. [9, 16] have quantitatively investigated the effects of camera movement speed on the level of VR sickness. Their results indicate that users start experiencing VR sickness when the speed is larger than 3 m/s.

Camera Acceleration: VR sickness can arise from excessive camera accelerating or decelerating movements [12, 17]. Recently, Terenzi et al. [18] have determined acceleration thresholds at which participants start to feel temporary symptoms of cybersickness.

Camera Rotation Axes: Users experience greater discomfort when they are viewing a VR scene with rotational camera movements compared with translational movements [19]. The severity of discomfort escalates significantly when the rotation movement involves more than one axis [20].

Field of View: The field of view (FOV) is the extent of the observable game world that is seen on display at any given moment. Many studies [21, 22] have attributed the VR sickness to an overly wide FOV setting (e.g., > 90 degrees). Restricting the content FOV can be a very effective measure to relieve both subjective and objective symptoms of VR sickness [23].

Depth of View: Carnegie et al. [24] investigated the effects of depth of view on VR sickness. They implemented a real-time dynamic depth of view (DOF) technique that kept the center of the screen in focus and reported that the usage of DOF blurring could reduce visual discomfort.

2.2 Assessing Risk Factors in VR Contents

Research has been done on assessing the degree of VR sickness via analyzing the VR content. This may avoid time-consuming physiological signal measurements [25]. Some VR game stores already adopt this approach: they display a comfort rating for each game (on a scale of *Comfortable*, *Moderate*, and *Intense*). These ratings are determined by human experts, who search for well-known risk factors inside the game content. Unfortunately, this manual assessment is highly labor-intensive and not scalable with the growing VR industry. To address this issue, a number of studies attempt to use machine learning for automatic VR sickness assessment. For example, Kim et al. [26] first proposed an autoencoder model to detect excessive scene movement in VR content and estimate the degree of motion sickness. They later proposed using a long short-term memory neural network to analyze exception camera movement and graphic complexity [27]. Padmanaban et al. [7] attempted a bagged decision tree model to identify risk factors in VR videos such as fast movement and overly wide FOV. Lee et al. [28, 29] utilized a deep convolutional neural network to analyze movement speed and depth disparity. A more recent study [30] used a deep learning-based framework to identify camera rotation movement in VR videos. Despite the progress made, many of these studies admit that a large collection of labeled data for real-world games is required to improve their model generalization [7, 26]. However, providing a scalable way to collect such a dataset is still an open challenge.

2.3 Training Data Acquisition

Existing works mainly attempted two data acquisition strategies: 1) Manual labeling and processing and 2) Purpose-built game contents.

Examples of manual labeling and processing include [25, 29]. They collect 360-degree videos from online video-sharing platforms, which typically contain scenes captured from a slow-driving car or drone. These were manually examined by human experts and assigned with risk factor labels such as fast/slow movement or rapid/gentle rotation. Several works [27] further attempted to extend the data size by post-processing techniques on the videos, such as using down/up sampling to simulate slow/fast motion. However, this approach cannot generate an extensive dataset because it is cumbersome and labor-intensive.

Regarding the purpose-built game contents, Stefan et al. [6] implemented a program that allows users to create custom roller coasters. Users can ride the coasters to gather video frames and the corresponding game engine data such as camera movement speed and acceleration. Similarly, several studies [7, 12] used the Unity 3D engine to create a reference scene that users can navigate freely. Many parameters of the scene can be adjusted, such as rotation degree or background textures. Unfortunately, the synthetic data distribution may not accurately reflect motions seen in real-world

VR games. This could cause generalizability issues when used for training machine learning models.

In this paper, we present a novel alternative; a data collection tool that can generate extensive annotated datasets from a wide range of real-world VR games without accessing their source code. The datasets created can be used for better machine learning training for recognizing the likelihood of motion sickness.

3 'VRHOOK' IMPLEMENTATION

The core mechanism of our tool is to record graphics rendering pipeline data, in particular, each rendered frame and its transformation matrices. By analyzing the matrices, our tool can automatically assign content risk factor labels to video frames, which then can be used to train a machine learning pipeline. To facilitate the readers' understanding of our data collection tool, we provide background on 3D computer game graphics in Appendix.

3.1 Data Recording via Dynamic Hooking

We could easily integrate the recording functionality by modifying a game's source code. However, this is generally impractical because many games are proprietary and their source code is unavailable. To overcome this issue, we adopt a black-box (i.e., no source code required) software engineering technique named *Dynamic Hooking* [8]. This can alter or augment a program's behavior by injecting custom code (i.e., hooks) into run-time memory. In our context, the custom code will be responsible for graphics pipeline data recording. To apply dynamic hooking, we first need to locate code injection sites in the game program. A prospective candidate would be the entrance of several important functions in the game loop described in Appendix. As demonstrated in Listing 1, we could acquire the view matrices and projection matrices by injecting the logging logic into the prologue of the *SetViewMatrix* function and the *SetProjMatrix* function. Similarly, we could obtain the rendered frames by intercepting the *SubmitFrameToVRHeadset* function.

It should be noted that these candidate functions are merely conceptual. In a real-world 3D game program, these functions may have completely different names and parameter lists, depending on many factors such as programming languages, 3D engines, and developers' programming habits. Pinpointing the actual memory addresses of those functions would demand solid reverse engineering expertise and many labor hours. To avoid these issues, our system does not place hooks on game programs. Instead, our system targets low-level operating system graphics stacks, more specifically, *DirectX* in Windows [31] or *OpenGL* in Linux [32]. These stacks are designed to receive hardware-agnostic rendering commands from applications and translate them into GPU-specific instructions to achieve high-performance rendering. They are highly standardized and remain almost the same regardless of the games being run and machines being used. As a result, our tool can always place hooks on the same memory addresses in these stacks to extract data from many real-world VR games.

In this paper, we mainly showcase how to apply dynamic hooking on DirectX, considering that the majority of PC VR games only run on Windows. Despite this focus, the methodologies are highly generic and can be easily adapted to OpenGL with minimal effort.

3.2 Acquiring the MVP Transformation Matrices

DirectX provides a programmable pipeline for generating real-time 3D graphics. It follows the MVP rendering process discussed in Appendix and includes the following major stages:

(1) Input Assembler Stage: Applications supply geometry data to the pipeline, such as triangles, lines, and points.
(2) Vertex Shader Stage: The vertex shader stage processes vertices from the input assembler, performing per-vertex operations such as MVP transformations, skinning, and per-vertex lighting.
(3) Pixel Shader Stage: The pixel shader stage calculates the color and transparency of a pixel on the screen based on what the vertex shader passes in.
(4) Output-Merger Stage: The output-merger is the final stage of the DirectX pipeline. It generates the final rendered pixel color using a combination of pipeline states such as pixel shader output and the contents of render targets.

We pay particular attention to the vertex shader stage, to which applications must supply their MVP matrices. Programmatically speaking, applications will store the MVP matrices in a dedicated data structure named *Constant Buffer* and submit it to the GPU via a DirectX function called *ID3D11DeviceContext::VSSetConstantBuffers*. By placing a hook on the function, we then can record all the constant buffers for each frame.

The immediate step is to extract the MVP matrices from the captured constant buffers. However, this can be tricky since different applications may store matrices in constant buffers using different memory layouts. What makes the matter worse is that applications can also store other types of shader parameters (e.g., color space) in the buffers as well. Our tool provides a reliable solution for locating the MVP matrices. We exploit the fact that most 3D game engines store the MVP matrices in some special constant buffers and expose them as global variables to simplify vertex/pixel shader

Listing 1 Conceptual Code Injection Points

```
1:  procedure SetViewMatrix(V)
2:      LogViewMatirx(V)        ▷ Injected logic to record the view matrices to a file.
3:      Do_SetViewMatrix_origin(V)   ▷ Execute original code of SetViewMatrix
4:  end procedure
5:  procedure SetProjMatrix(P)
6:      LogProjMatirx(P)        ▷ Injected logic to record the project matrices to a file.
7:      Do_SetProjMatrix_origin(P)   ▷ Execute original code of SetProjMatrix
8:  end procedure
9:  procedure SubmitFrameToVRHeadset(F)
10:     LogRenderedFrame(F)     ▷ Injected logic to record the rendered frames to a file.
11:     Do_SubmitFrameToVRHeadset_origin(F)   ▷ Execute original code of SubmitFrameToVRHeadset
12: end procedure
```

development [33]. These buffers are automatically activated and have unique signatures and fixed memory layouts[1]. For instance, the Unity and Unreal engines will always pack the MVP matrices in constant buffers with a distinguishable size of 368 byte or 192 byte, respectively. As a result, we only need to capture constant buffers with these sizes and extract the MVP matrices from predetermined memory offsets. To test the effectiveness of this approach, we tested our tool on 127 top-selling games from Steam store. Among them, 81% are implemented using the Unity engine, while 17% are built on top of the Unreal engine. Our approach successfully identified and captured MVP matrices from these games' constant buffers. Considering the Unity and Unreal engines possess very high market share in the VR gaming industry, we argue this signature-based method to be sufficiently generalizable.

3.3 Capturing Rendered Frames

DirectX adopts a *Swap Chain* mechanism to display rendered frames on screen. Specifically, DirectX defines the GPU buffer currently being displayed on the monitor as the front buffer. Instead of drawing directly on that buffer, an application renders images onto an auxiliary GPU buffer, called the back buffer. When the drawing on the back buffer is done, the application can invoke a DirectX function named *IDXGISwapChain::Present* to update the front buffer with the contents of the back buffer. This buffer swapping design can alleviate the timing conflict between software and hardware, thus preventing graphical artifacts appearing on the screen.

Our tool installs a hook on the *IDXGISwapChain::Present* method to intercept the back buffer pointer before swapping. However, the back buffer resides in GPU memory and cannot be directly accessed by CPU. A workaround is to transfer the back buffer contents from GPU memory to the system memory and access them with virtual address mapping. This can be achieved with two DirectX commands; *CopySubresourceRegion* and *Map*. Careful consideration should be given to this approach, as it may incur a significant performance bottleneck. Specifically, GPUs and CPUs are cooperating in an asynchronous fashion. When a copy operation is initiated by the CPU, the contents in GPU memory are not immediately transferred to system memory. If our application tries to access the results of the copy operation before it is finished, the GPU is forcibly stalled in order to synchronize with the CPU. This can lead to a severe frame rate drop and render the game unplayable. To prevent this, our tool maintains three staging buffers and rotates between them in the following manner:

(1) Frame N: copy Frame N to staging buffer 1.
(2) Frame $N+1$: copy Frame $N+1$ to staging buffer 2.
(3) Frame $N+2$: copy Frame $N+2$ to staging buffer 3 and map staging buffer 1 to access data of Frame N.

This approach would not affect the game's frame rate or latency. It introduces approximately 50 ms delay to our backend data logging system. This is acceptable since our tool is not designed for real-time data processing.

Our tool also supports capturing per-eye frames in VR headsets. This is implemented by placing hooks into OpenVR [34], a low-level runtime library that allows VR games to access VR hardware from multiple vendors in a uniform manner. In detail, OpenVR provides an interface for displaying a rendered image on a VR headset, namely, *IVRCompositor::Submit*. The function accepts a VR headset screen index (i.e., left eye or right eye) and a pointer to a GPU buffer. We can intercept and capture the buffer in the same manner as we do for the back buffer.

Optionally, our tool can capture the depth buffer associated with each rendered frame. This is achieved by injecting additional rendering instructions at the output-merger stage. In brief, we first allocate a depth-stencil texture with the same dimension as the back buffer. During the output-merger stage, we will bind the texture as a render target to enable calculation of depth information for each pixel. After that, the depth buffer can be retrieved from GPUs for post-processing.

3.4 Generating Labels for Risk Factors

From the captured frames and transformed matrices, our tool is able to deducible several risk factor as follows:

Camera/Object Speed: We can extract camera and object location coordinates from view and model matrices respectively. By taking the first-order of these coordinates, we can infer their movement velocities. We can then apply thresholds suggested by previous studies [9] to label frames with excessive speed.

Camera/Object Acceleration: We can infer camera/object acceleration by taking the second-order derivative of their location coordinates. We then adopt thresholds informed by previous works [18] to label frames with excessive acceleration.

Camera Rotation Axes: Given two consecutive view matrices V_1 and V_2, we can calculate the camera rotation difference via the formula: $V_\Delta = V_2 * {V_1}^{-1}$, where V_Δ can be further decomposed into 3 elementary Euler angles, commonly referred to yaw, roll, and pitch. If any two of them exceed a predefined value [19], we assign multi-axis rotation labels to the frames.

Field Of View: We can extract FOV values from projection matrices. We can then generate binary labels (i.e., wide/narrow) using thresholds suggested by previous research [23].

Depth of View: Our tool can obtain DOV values for each frame. This is achieved by capturing the Z-buffer associated with each rendered frame. The buffer records the depth value of every pixel inside the viewing frustum.

3.5 Collecting Self-reported Data from Users

Motivated by previous research [11], our tool implements a measurement tool to enable users to record their feeling of cybersickness at any moment. Specifically, it provides users with the Microsoft Surface Dial[2] to continuously report their level of cybersickness. When the dial is first pressed down, a feedback slider (shown in Figure 2(a)) will appear below the direct line of sight. When the dial is turned clockwise, the slider value increases to a maximum of 5, indicating the scene results in severe discomfort. If turned anti-clockwise, the value decreases to a minimum of 1, indicating the current scene does not incur motion sickness. An emoji icon,

[1]We can obtain the memory layouts from the engines' reference shader source codes. https://docs.unity3d.com/Manual/SL-BuiltinIncludes.html

[2]https://www.microsoft.com/en-nz/d/surface-dial

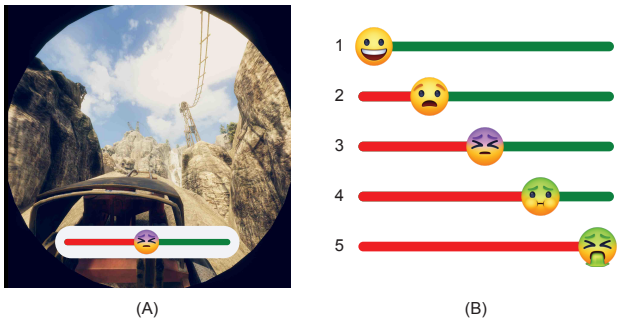

Figure 2: UI for Collecting Self-Reported Comfort Ratings

sitting on top of the slider, changes according to the selected comfort score as shown Figure 2(b). The dial also provides oscillating vibration as feedback when it is turned. The user can confirm their choice by pressing the dial again. The values are then recorded with corresponding timestamps into a CSV file. This approach is not only accurate but also keeps the interference with the actual VR task users perform at a minimum [11].

One essential step to implementing this measurement tool is to display a slider overlay on top of the current game scene without needing the access to source code. Our tool achieves this by inserting custom rendering commands before the back buffer is swapped. In detail, we first create an additional offscreen rendering texture. We then bind this texture as a render target and issue all the rendering commands for the slider interface. Finally, we blend the offscreen texture with the back buffer for overlay display. Compared with direct rendering to the back buffer, this approach is slightly more efficient since we can reuse rendered textures and prevent repeated GPU computation on every frame. Another technical issue is to retrieve the input events from the Microsoft Surface Dial. Although Microsoft has provided a set of APIs to interact with this hardware, these APIs require the Universal Windows Platform (UWP) run-time framework to be loaded. A considerable number of games do not meet the requirements. To circumvent this issue, we adopt a low-level human interface device (HID) driver [35] to poll the raw input events from the hardware directly.

4 UTILIZATION OF THE 'VRHOOK' TOOL

Our VRhook tool can generate extensive annotated datasets from real-world VR games. Therefore, it can facilitate a wide range of machine learning tasks. To demonstrate VRhook's utility, we build a proof-of-concept machine learning system to detect risk factors and estimate a comfort rating given a VR gameplay video.

4.1 System Architecture

Our system features a two-stage pipeline, as shown in Fig. 7. The first stage takes a video stream as input and detects the presence of three major risk factors: 1) excessive camera speed, 2) excessive camera acceleration, and 3) multi-axis camera rotation. We adopt *SlowFast* [36], a state-of-the-art self supervised learning model for video recognition. Self-supervised learning conducts model pre-training to learn one part of the input from another part of the input. For instance, given the upper half of the video frame, predict the lower half of the same frame. By doing this, the model

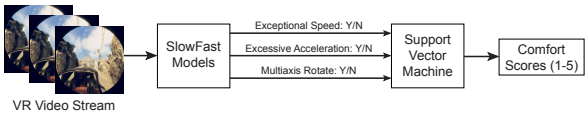

Figure 3: A cascading machine learning pipeline for risk factor detection and comfort rating estimation.

can learn useful representations of input videos and later fine-tune them with a limited number of labels for actual supervised tasks. Self-supervised learning generally outperforms conventional supervised learning, particularly when the training dataset is small [37]. It is worthwhile to mention that SlowFast already provides a model checkpoint pre-trained on sizeable video datasets. Therefore, we could fine-tune the checkpoint for three labeled datasets (i.e., speed, acceleration, and rotation axis) from our tool to obtain three separate models. They will process the video stream in a sliding window of one-second and output 3 binary numbers to indicate the presence of risk.

The second stage analyzes the first-stage output and predicts a comfort rating on a scale of 1 to 5 (a higher number corresponds to a higher level of discomfort). We apply a buffer on the first-stage output and process it every 4 seconds. In other words, the input to the second-stage model is a 3×4 matrix. This choice is motivated by previous research [11], which suggests a 4-second time span is sufficient for users to interpret VR content and generate a momentary cybersickness response. Considering the simple form of the input/output data, we decided to apply a support vector machine (SVM) at this stage, a simple-yet-effective machine learning model.

4.2 Data Collection

Risk Factor Labels: To train the first-stage pipeline, we used our tool to collect a dataset from 5 real-world roller coaster games. On average each game lasted for 4 minutes. In total, we gathered approximately 1200 one-second video clips. Each clip contained 180 video frames (90 for each eye) at a resolution of 1832 × 960. These frames' MVP matrices were recorded and analyzed to generate three types of binary labels for each clip, including 1) fast/slow speed, 2) fast/slow acceleration, and 3) multiple-axis rotation detected/not detected.

We can easily increase the data size by exploiting the fact that in a VR game, the view matrix (also the video frame) is adjusted according to the VR headset orientation. The orientation is described by three values: yaw (-180 deg to 180 deg), roll (-180 deg to 180 deg), and pitch (-90 deg to 90 deg). By using a step size of 30 degrees for each value, we constructed 864 configurations. For each configuration, we programmatically changed the headset orientation and generated a new independent dataset. Eventually, we captured 1200×864 one-second video clips.

Comfort Rating: To train the SVM model in the second pipeline stage, we conducted a study to collect users' comfort rating scores on the five VR games. The study was approved by The University of Auckland ethics Committee. We recruited 16 participants (6 Female, 10 Male, age M = 27, SD = 3). At the start of the experiment, they were first provided the participant information, and then shown how they could report the momentary sickness level using the Surface Dial.

During the game sessions, we set our tool to prompt the user every 30 seconds. Participants could also voluntarily open the prompt

and report at any time by pressing down the dial. They were encouraged to report whenever they felt a change in their discomfort level. They were allowed stop participating for any reason, including feeling overwhelmed. On average, each participant attempted 1.4 (±0.7) games. For each game session, on average, a participant reported 23 (±4) comfort rating scores. We assigned each user rating to a 4-second gameplay video clip right before the report. If multiple users provided ratings for the same clip, we then used the average score. Among all the clips with more than 7 user ratings, the highest standard deviation was 0.70. In the end, we gathered 178 video clips and the corresponding comfort rating scores.

4.3 System Evaluation

We first split our dataset into training and test sets in a ratio of 80/20 (i.e., four games for training and the remaining game for testing). We utilized the PyTorchVideo [38] framework to train the SlowFast model. We set the number of epochs to 5 and the batch size to 32. The learning rate was $1e^{-4}$. We used two V100 32 GB GPUs and the training wall time was 12 hours. The performance of our system is shown in Table 1. It can be seen that our system can predict the presence of three common risk factors and estimate comfort ratings with reasonable accuracy. An interesting observation is that our pipeline achieves higher accuracy (6%) at detecting excessive speed than multi-axis rotation. This might be attributed to the internal mechanism of the SlowFast model. SlowFast is pre-trained using a two-path model. One pathway is designed to capture semantic information and it operates at low frame rates and slow refreshing speed. The other pathway is responsible for capturing rapidly changing motion, by operating at a fast refreshing speed and high temporal resolution. As a result, SlowFast is more sensitive to differences in movement speeds than rotation degrees.

The main purpose of our ML pipeline is to demonstrate how VRhook can be used rather than provide the highest absolute accuracy. Nevertheless, we suggest several potential approaches to boost the performance. A low-hanging fruit is to increase the dataset size. The expected workload is very manageable since our tool enables automatic labeled data acquisition from almost every off-the-shelf VR game. Alternatively, we could attempt more advanced parameter tuning approaches. For instance, we may split our dataset into training, validation, and test sets in a ratio of 70/15/15. Having an extra validation set would allow us to choose a set of more performant hyperparameters for our models. Lastly, instead of using the existing SlowFast checkpoint, we could further pre-train the SlowFast model on our input videos so that it can learn more accurate data representations for VR contents.

5 DISCUSSION

5.1 Stimulating Machine Learning Research

Our tool aims to stimulate new machine learning research in VR sickness by providing convenient access to extensive labeled data from real-world games. In Section 4, we showcased a machine learning pipeline to detect common risk factors in VR contents and estimate the comfort rating. Besides this use case, we are investigating two other application scenarios. Firstly, we are exploring the potential of the depth buffer associated with each frame. From the depth information, we can generate motion (optical) flow, which encodes abundant temporal information (e.g., speed, acceleration, and rotation) for a game scene. The motion flow may then be analyzed by a deep neural network for comfort rating prediction [39]. Secondly, our tool can be extended to capture various types of sensory readings (e.g., headset movement, controller inputs, heart rate, or ECG). This would benefit many cross-disciplinary studies (e.g., [27, 40]) that combine physiological signals and content features to predict cybersickness.

5.2 Implication to VR Industry

Our pipeline could benefit online VR game stores. For example, our system could automatically assign a provisional comfort rating to a game by examining its demo video. Based on the rating, human experts can decide whether to examine the game contents in detail. In addition, the detected risk factors from the first pipeline stage could be reported to developers so that they can adjust the game level design accordingly for a safer gaming experience. It should be noted that the ML pipeline shown in Section 4 is only one use case of our data collection tool.

5.3 Alternative Machine Learning Architectures

Our pipeline in Section 4 uses two sequential components for risk factor detection and comfort rating prediction. Alternatively, we could adapt a single deep neural network that outputs risk factors and comfort rating scores in parallel. Compared to the sequential design, this architecture would require more self-reported data for training; for each input video clip, one user feedback is required. In return, the network may deliver a higher performance since it now can learn more latent relationships between video content and comfort rating. Our tool can support both paradigms and developers could select one that suits their application requirements best. In addition, our pipeline fine-tunes three SlowFast models separately. This allows us to add more risk factors and extend the pipeline in the future without retraining the existing models. Recent studies [41] suggest that if multiple tasks share information, training them together in one model may achieve higher performance. We plan to investigate this training paradigm in the future.

5.4 OpenGL and OpenXR

In this paper, we demonstrate data collection methodologies for DirectX in Windows, considering its dominant market share. Nevertheless, our tool provides experimental support for OpenGL in Linux and macOS. In fact, the required effort is minimal since many DirectX primitives have direct correspondences in OpenGL. For example, in analogy to DirectX's constant buffers, OpenGL uses uniform buffer objects to provide storage for MVP matrices. OpenGL also applies a swap chain to present rendered images to an application view or a monitor screen. Our tool installs hooks on the OpenVR runtime library to intercept VR joystick input signals and per-eye favor video frames. Recently, a small number of games are experimenting with a newer runtime library called OpenXR [42]. Compared to OpenVR, OpenXR provides higher performance access to diverse VR devices across multiple platforms. Note that OpenVR is not deprecated. Instead, OpenVR is still the default runtime library for most recent VR games. Nevertheless, to be future-proof, we have integrated preliminary support to OpenXR in our tool.

Table 1: Pipeline Performance

First Pipeline Stage:		
Risk Factors	Accuracy	F1 Score
Excessive Speed	0.76	0.71
Excessive Acceleration	0.78	0.75
Multi-axis Rotation	0.78	0.76
Second Pipeline Stage:		
SVM Accuracy	Weighted F1 Score	
0.70	0.69	

5.5 Limitations

Our proof of concept machine learning pipeline was validated on a small number of games and participants. However, the main intention of this paper was to showcase the utilization of our data collection tool. The very aim of our tool is to facilitate researchers easily collecting data to support machine learning research. In our future work, we too plan to collect more data and explore more machine learning techniques. In addition, our tool is not ready to generate some less-known risk factor labels such as audio effects [43] and graphic realism [44]. More research effort on them is warranted.

6 CONCLUSION

In this paper, we present VRhook, a novel data collection tool that can collect labeled data from real-world VR games to facilitate machine learning research in VR sickness. Our tool does not require access to game source code. Instead, it uses dynamic hooking and injects custom code into run-time memory to capture low-level graphics pipeline data. We can assign each rendered frame with useful labels such as rotation, speed, and acceleration with them. In addition, our tool can blend a customized screen overlay on top of games to collect self-reported comfort scores. We showcase how our tool can be used to build a machine learning pipeline for VR sickness risk factor detection and comfort rating estimation given a VR gameplay video.

7 ACKNOWLEDGMENT

This work was supported by Meta research grant via ARIVE network.

REFERENCES

[1] "Virtual reality in gaming market size." [Online]. Available: https://www.fortunebusinessinsights.com/industry-reports/virtual-reality-gaming-market-100271

[2] S. Martirosov and P. Kopecek, "Cyber sickness in virtual reality-literature review." *Annals of DAAAM & Proceedings*, vol. 28, 2017.

[3] D. M. Johnson, "Introduction to and review of simulator sickness research," 2005.

[4] E. Chang, H. T. Kim, and B. Yoo, "Virtual reality sickness: a review of causes and measurements," *International Journal of Human–Computer Interaction*, vol. 36, no. 17, pp. 1658–1682, 2020.

[5] H. Oh and W. Son, "Cybersickness and its severity arising from virtual reality content: A comprehensive study," *Sensors*, vol. 22, no. 4, p. 1314, 2022.

[6] S. Hell and V. Argyriou, "Machine learning architectures to predict motion sickness using a virtual reality rollercoaster simulation tool," in *2018 IEEE International Conference on Artificial Intelligence and Virtual Reality (AIVR)*. IEEE, 2018, pp. 153–156.

[7] N. Padmanaban, T. Ruban, V. Sitzmann, A. M. Norcia, and G. Wetzstein, "Towards a machine-learning approach for sickness prediction in 360 stereoscopic videos," *IEEE transactions on visualization and computer graphics*, vol. 24, no. 4, pp. 1594–1603, 2018.

[8] J. Lopez, L. Babun, H. Aksu, and A. S. Uluagac, "A survey on function and system call hooking approaches," *Journal of Hardware and Systems Security*, vol. 1, no. 2, pp. 114–136, 2017.

[9] R. H. So, A. Ho, and W. Lo, "A metric to quantify virtual scene movement for the study of cybersickness: Definition, implementation, and verification," *Presence*, vol. 10, no. 2, pp. 193–215, 2001.

[10] F. Bonato, A. Bubka, S. Palmisano, D. Phillip, and G. Moreno, "Vection change exacerbates simulator sickness in virtual environments," *Presence: Teleoperators and Virtual Environments*, vol. 17, no. 3, pp. 283–292, 2008.

[11] N. McHugh, S. Jung, S. Hoermann, and R. W. Lindeman, "Investigating a physical dial as a measurement tool for cybersickness in virtual reality," in *25th ACM Symposium on Virtual Reality Software and Technology*, 2019, pp. 1–5.

[12] H. K. Kim, J. Park, Y. Choi, and M. Choe, "Virtual reality sickness questionnaire (vrsq): Motion sickness measurement index in a virtual reality environment," *Applied ergonomics*, vol. 69, pp. 66–73, 2018.

[13] D. Saredakis, A. Szpak, B. Birckhead, H. A. Keage, A. Rizzo, and T. Loetscher, "Factors associated with virtual reality sickness in head-mounted displays: a systematic review and meta-analysis," *Frontiers in human neuroscience*, vol. 14, p. 96, 2020.

[14] J.-R. Chardonnet, M. A. Mirzaei, and F. Mérienne, "Visually induced motion sickness estimation and prediction in virtual reality using frequency components analysis of postural sway signal," in *International conference on artificial reality and telexistence eurographics symposium on virtual environments*, 2015, pp. 9–16.

[15] C.-L. Liu and S.-T. Uang, "A study of sickness induced within a 3d virtual store and combated with fuzzy control in the elderly," in *2012 9th International Conference on Fuzzy Systems and Knowledge Discovery*. IEEE, 2012, pp. 334–338.

[16] R. H. So, W. Lo, and A. T. Ho, "Effects of navigation speed on motion sickness caused by an immersive virtual environment," *Human factors*, vol. 43, no. 3, pp. 452–461, 2001.

[17] B. Keshavarz, B. E. Riecke, L. J. Hettinger, and J. L. Campos, "Vection and visually induced motion sickness: how are they related?" *Frontiers in psychology*, vol. 6, p. 472, 2015.

[18] L. Terenzi and P. Zaal, "Rotational and translational velocity and acceleration thresholds for the onset of cybersickness in virtual reality," in *AIAA Scitech 2020 Forum*, 2020, p. 0171.

[19] F. Bonato, A. Bubka, and S. Palmisano, "Combined pitch and roll and cybersickness in a virtual environment," *Aviation, space, and environmental medicine*, vol. 80, no. 11, pp. 941–945, 2009.

[20] B. Keshavarz and H. Hecht, "Axis rotation and visually induced motion sickness: the role of combined roll, pitch, and yaw motion," *Aviation, space, and environmental medicine*, vol. 82, no. 11, pp. 1023–1029, 2011.

[21] H.-L. Duh, J. Lin, R. V. Kenyon, D. E. Parker, and T. A. Furness, "Effects of field of view on balance in an immersive environment," in *Proceedings IEEE Virtual Reality 2001*. IEEE, 2001, pp. 235–240.

[22] J.-W. Lin, H. B.-L. Duh, D. E. Parker, H. Abi-Rached, and T. A. Furness, "Effects of field of view on presence, enjoyment, memory, and simulator sickness in a virtual environment," in *Proceedings ieee virtual reality 2002*. IEEE, 2002, pp. 164–171.

[23] I. B. Adhanom, N. N. Griffin, P. MacNeilage, and E. Folmer, "The effect of a foveated field-of-view restrictor on vr sickness," in *2020 IEEE conference on virtual reality and 3D user interfaces (VR)*. IEEE, 2020, pp. 645–652.

[24] K. Carnegie and T. Rhee, "Reducing visual discomfort with hmds using dynamic depth of field," *IEEE computer graphics and applications*, vol. 35, no. 5, pp. 34–41, 2015.

[25] P.-C. Kuo, L.-C. Chuang, D.-Y. Lin, and M.-S. Lee, "Vr sickness assessment with perception prior and hybrid temporal features," in *2020 25th International Conference on Pattern Recognition (ICPR)*. IEEE, 2021, pp. 5558–5564.

[26] H. G. Kim, W. J. Baddar, H.-t. Lim, H. Jeong, and Y. M. Ro, "Measurement of exceptional motion in vr video contents for vr sickness assessment using deep convolutional autoencoder," in *Proceedings of the 23rd ACM Symposium on Virtual Reality Software and Technology*, 2017, pp. 1–7.

[27] K. Kim, S. Lee, H. G. Kim, M. Park, and Y. M. Ro, "Deep objective assessment model based on spatio-temporal perception of 360-degree video for vr sickness prediction," in *2019 IEEE International Conference on Image Processing (ICIP)*. IEEE, 2019, pp. 3192–3196.

[28] T. M. Lee, J.-C. Yoon, and I.-K. Lee, "Motion sickness prediction in stereoscopic videos using 3d convolutional neural networks," *IEEE transactions on visualization and computer graphics*, vol. 25, no. 5, pp. 1919–1927, 2019.

[29] S. Lee, S. Kim, H. G. Kim, M. S. Kim, S. Yun, B. Jeong, and Y. M. Ro, "Physiological fusion net: Quantifying individual vr sickness with content stimulus and physiological response," in *2019 IEEE International Conference on Image Processing (ICIP)*. IEEE, 2019, pp. 440–444.

[30] S. Kim, S. Lee, and Y. M. Ro, "Estimating vr sickness caused by camera shake in vr videography," in *2020 IEEE International Conference on Image Processing (ICIP)*. IEEE, 2020, pp. 3433–3437.

[31] F. Luna, *Introduction to 3D game programming with DirectX 11*. Stylus Publishing, LLC, 2012.

[32] D. Hearn, M. P. Baker, and M. P. Baker, *Computer graphics with OpenGL*. Pearson Prentice Hall Upper Saddle River, NJ:, 2004, vol. 3.

[33] M. Menard and B. Wagstaff, *Game development with Unity*. Course Technology, 2012.

[34] S. OpenVR, "Url: https://github. com," *ValveSoftware/openvr (visited on 02/02/2019)*.

[35] E. Verling, "Development of a user-space application for an hid device, using libhid," *Linux Journal*, no. 138, pp. 42–46, 2005.
[36] C. Feichtenhofer, H. Fan, J. Malik, and K. He, "Slowfast networks for video recognition," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2019, pp. 6202–6211.
[37] X. Liu, F. Zhang, Z. Hou, L. Mian, Z. Wang, J. Zhang, and J. Tang, "Self-supervised learning: Generative or contrastive," *IEEE Transactions on Knowledge and Data Engineering*, 2021.
[38] H. Fan, T. Murrell, H. Wang, K. V. Alwala, Y. Li, Y. Li, B. Xiong, N. Ravi, M. Li, H. Yang *et al.*, "Pytorchvideo: A deep learning library for video understanding," in *Proceedings of the 29th ACM International Conference on Multimedia*, 2021, pp. 3783–3786.
[39] M. Du, H. Cui, Y. Wang, and H. Duh, "Learning from deep stereoscopic attention for simulator sickness prediction," *IEEE Transactions on Visualization and Computer Graphics*, 2021.
[40] D. K. Jeong, S. Yoo, and Y. Jang, "Vr sickness measurement with eeg using dnn algorithm," in *Proceedings of the 24th ACM Symposium on Virtual Reality Software and Technology*, 2018, pp. 1–2.
[41] C. Fifty, E. Amid, Z. Zhao, T. Yu, R. Anil, and C. Finn, "Efficiently identifying task groupings for multi-task learning," *Advances in Neural Information Processing Systems*, vol. 34, pp. 27 503–27 516, 2021.
[42] M. S. Brennesholtz, "3-1: Invited paper: Vr standards and guidelines," in *SID Symposium Digest of Technical Papers*, vol. 49, no. 1. Wiley Online Library, 2018, pp. 1–4.
[43] O. X. Kuiper, J. E. Bos, C. Diels, and E. A. Schmidt, "Knowing what's coming: Anticipatory audio cues can mitigate motion sickness," *Applied Ergonomics*, vol. 85, p. 103068, 2020.
[44] J. F. Golding, K. Doolan, A. Acharya, M. Tribak, and M. A. Gresty, "Cognitive cues and visually induced motion sickness," *Aviation, space, and environmental medicine*, vol. 83, no. 5, pp. 477–482, 2012.

APPENDIX: 3D GRAPHICS RENDERING PIPELINE

Model Space: In 3D graphics engines, scenes are typically represented as models in three-dimensional space, with each model composed of many vertices. For instance, if we define a model with eight vertices: (1,1,1), (1,1,-1), (1,-1,1), (1,-1,-1), (-1,1,1), (-1,1,-1), (-1,-1,1), and (-1,-1,-1), we would obtain a $2 \times 2 \times 2$ cube geometry centered at (0,0,0) as shown in Figure 4 (a). Often in a scene, a certain geometry may appear multiple times, but with different locations, sizes, or rotations as shown in Figure 4 (b). Creating models with unique vertices for each instance can be memory intensive. Instead, a single set of geometry vertices is shared across multiple instances, but with each instance applying its own unique set of model transformations. A model transformation matrix, denoted as M, can be represented as follows:

$$M = TRS, \tag{1}$$

where T denotes the translation transform, R represents the rotation transform, and S describes the scale transform. When multiplying the matrix to a model vertex, we will obtain a vertex in world space:

$$v_{world} = Mv_{model}. \tag{2}$$

World space is a shared global 3D Cartesian coordinate system. All renderable models exist within this space, arranged by their model matrix relative to the same (0,0,0) point.

Camera Space: Cameras are a special object in world space. In computer graphics, all rendering is from a camera's perspective (in analogy to a real-life camera). Each camera has a model matrix

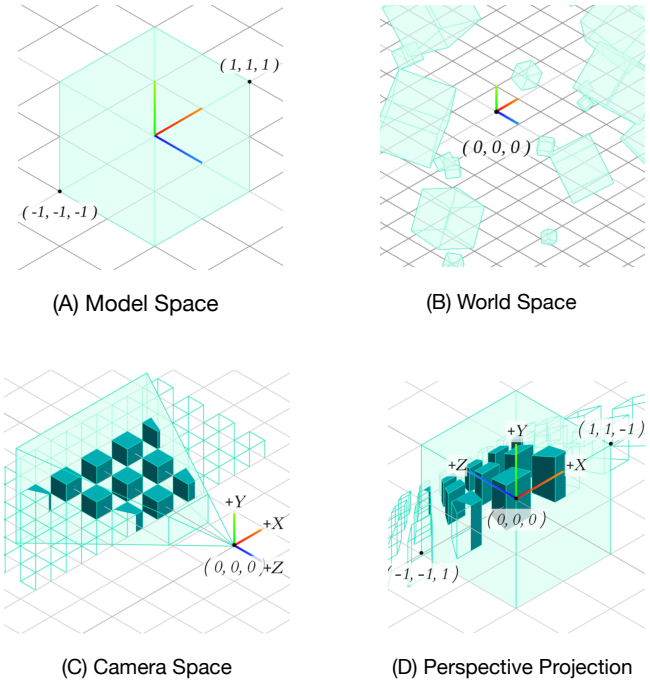

Figure 4: Model, World, Camera, and Screen Space (Adapted from bit.ly/3DMI9oL with permission).

C defining its position and orientation in world space. We define the inverse of the camera's model matrix as the view matrix, i.e., $V = C^{-1}$. The view matrix can then transform vertices from the world space to the camera space:

$$v_{camera} = Vv_{world} = VMv_{model}. \tag{3}$$

In this space, all vertices are defined in a coordinate system where the camera is sitting at the origin (0,0,0) and facing down its -Z axis[3] as demonstrated in Figure 4 (c). Note that in a VR game, two cameras are used to render a different perspective for the user's left and right eye to create stereoscopic effect.

Screen Space: Once vertices are in camera space, they can finally be transformed into two-dimensional screen space by using a projection transformation. In analogy to the lenses of a conventional camera, the transformation can decide how much of the scene is captured on the screen by adjusting the field of view. Figure 4 (d) showcases a commonly used perspective projection, which creates a natural effect where objects farther away from viewers appear smaller. If we denote the projection matrix as P, we can obtain vertices in the screen space by using the following equation:

$$v_{screen} = Pv_{camera} = PVMv_{model}. \tag{4}$$

The latter part of this equation is referred to as the Model-View-Projection (MVP) transform.

Listing 2 Pesudo Code for Game Loop

```
1:  procedure GameLoop ▷ Invoked repeatedly until programs exit.
2:      Input ← RecvUserInput ▷ Receive user input.
3:      Ms, V, P ← UpdateMatrices(Input) ▷ Calculate MVP matrices based on user input and game logic.
4:      for each Obj ∈ Scene do
5:          SetModelMatrix(Obj, Ms) ▷ Set model matrices for each object in the current scene.
6:      end for
7:      SetViewMatrix(V) ▷ Set view matrices.
8:      SetProjMatrix(P) ▷ Set projection matrices.
9:      frame ← RenderOnGPU() ▷ Start GPU Rendering.
10:     SubmitFrameToVRHeadset(frame) ▷ Display the newly rendered frame on a VR headset.
11: end procedure
```

Game Loop: A game loop, as described in Algorithm 2, is a continuous controlled loop that keeps the game running. This game loop is the heart of almost every 3D game. It comprises three main sections: 1) receive user input, 2) update MVP matrices based on user input and game states, and 3) render image frames on the GPU. Generally the game loop will run at 60 to 120 frames per second. VR games need an extra step in the game loop: copying newly-rendered frames from the GPU to the VR headset hardware for display.

,

[3]We follow the right-handed OpenGL convention.